



INTERNNOVA - JAVA INTERNSHIP

WEEK 3 - ASSIGNMENT



Submitted By : Subhasri S

Branch : Computer Science and Engineering

College : PSNA College of Engineering and Technology

Internship Domain : Java Development

Assignment : Week 3 – Object-Oriented Programming

Submission Date : 15 August 2026

OBJECTIVE

The objective of this assignment is to understand and implement the fundamental concepts of Object-Oriented Programming in Java. The assignment provides practical experience in creating classes and objects and using important OOP concepts to develop structured and reusable programs.

This assignment focuses on classes and objects, constructors, the `this` and `static` keywords, inheritance, polymorphism, encapsulation, and abstraction. These concepts are essential for developing modular, maintainable, and reusable Java applications.

Through practical implementation, the assignment helps in understanding how objects represent real-world entities and how classes can be used to organize data and functionality. It also demonstrates how inheritance and polymorphism improve code reusability and flexibility.

The assignment further introduces data protection through encapsulation and the concept of hiding implementation details using abstraction. By implementing these concepts through separate programs, the relationship between different OOP features can be understood more clearly.

LEARNING OBJECTIVES

After completing this assignment, the following concepts are expected to be understood:

1. Classes and Objects – Create classes and instantiate multiple objects to represent data.
2. Constructors – Understand default and parameterized constructors and their role in object initialization.
3. `this` and `static` – Understand the use of the current object reference and class-level variables.
4. Inheritance – Reuse properties and methods of a parent class through child classes.
5. Polymorphism – Implement method overloading and method overriding.
6. Encapsulation – Protect data using private variables and getter/setter methods.
7. Abstraction – Hide implementation details using abstract classes and methods

TASK 1: CLASSES & OBJECTS – STUDENT MANAGEMENT

Objective

To create a Student class with appropriate attributes and create multiple objects to store and display the information of different students.

Brief Explanation

This program demonstrates the basic concepts of classes and objects in Java. A Student class is created with attributes such as student name, roll number, branch, and CGPA. Multiple objects are created from the Student class to represent different students. The attributes of each object are accessed and displayed in a proper format.

Source Code

File Name: StudentManagement.java

```
class Student {  
    String studentName;  
    int rollNumber;  
    String branch;  
    double cgpa;
```

```
Student(String studentName, int rollNumber, String  
branch, double cgpa) {
```

```
    this.studentName = studentName;
```

```
    this.rollNumber = rollNumber;
```

```
    this.branch = branch;
```

```
    this.cgpa = cgpa;
```

```
}
```

```
void displayStudent() {
```

```
    System.out.println("Name      : " + studentName);
```

```
    System.out.println("Roll Number: " + rollNumber);
```

```
    System.out.println("Branch   : " + branch);
```

```
    System.out.println("CGPA     : " + cgpa);
```

```
    System.out.println();
```

```
}
```

```
}
```

```
public class StudentManagement {
```

```
    public static void main(String[] args)
```

```
    {  
        Student student1 = new Student("Subhasri S", 101,  
        "CSE", 9.18);
```

```
Student student2 = new Student("Sathish M", 102,  
"CSE", 9.00);
```

```
Student student3 = new Student("Vasundra M", 103,  
"CSE", 8.90);
```

```
System.out.println("Student Management");
```

```
System.out.println("-----");
```

```
System.out.println();
```

```
System.out.println("Student 1");
```

```
System.out.println("-----");
```

```
student1.displayStudent();
```

```
System.out.println("Student 2");
```

```
System.out.println("-----");
```

```
student2.displayStudent();
```

```
System.out.println("Student 3");
```

```
System.out.println("-----");
```

```
student3.displayStudent();
```

```
}
```

```
}
```

Output

```
● PS D:\Internova\Week_3\task_1> javac StudentManagement.java
● PS D:\Internova\Week_3\task_1> java StudentManagement
Student Management
-----

Student 1
-----
Name       : Subhasri S
Roll Number: 101
Branch     : CSE
CGPA       : 9.18

Student 2
-----
Name       : Sathish M
Roll Number: 102
Branch     : CSE
CGPA       : 9.0

Student 3
-----
Name       : Vasundra M
Roll Number: 103
Branch     : CSE
CGPA       : 8.9
```

TASK 2: CONSTRUCTORS – EMPLOYEE INFORMATION

Objective

To understand and implement default and parameterized constructors in Java for initializing employee objects.

Brief Explanation

A constructor is a special member of a class that is automatically called when an object is created. This program demonstrates both a default constructor and a parameterized constructor. The default constructor initializes an employee object with default values, while the parameterized constructor accepts values through parameters and uses them to initialize the employee object.

Source Code

File Name: EmployeeInformation.java

```
class Employee {  
    int employeeId;  
    String employeeName;  
    String department;  
    double salary;  
    Employee() {  
        employeeId = 0;
```

```

        employeeName = "Not Assigned";
        department = "Not Assigned";
        salary = 0.0;
    }

Employee(int employeeId, String
employeeName, String department, double salary)
{
    this.employeeId = employeeId;
    this.employeeName=employeeName;
    this.department = department;
    this.salary = salary;
}

void displayEmployee() {
    System.out.println("Employee ID  : "
+employeeId);
    System.out.println("Employee Name : " +
employeeName);
    System.out.println("Department   : " +
department);
    System.out.println("Salary       : " + salary);
    System.out.println();
}
}

public class EmployeeInformation {

```

```

    public static void main(String[] args){
        Employee defaultEmployee = new
Employee();
        Employee parameterizedEmployee =new
Employee(102, "Subhasri S", "Development",
30000.0);
        System.out.println("Employee Information");
        System.out.println("-----");
        System.out.println();
        System.out.println("Default Constructor");
        System.out.println("-----");
        defaultEmployee.displayEmployee();
        System.out.println("Parameterized
Constructor");
        System.out.println("-----");
        parameterizedEmployee.displayEmployee();
    }
}

```

Output

```
• PS D:\Internova\Week_3\task_2> javac EmployeeInformation.java
• PS D:\Internova\Week_3\task_2> java EmployeeInformation
Employee Information
-----

Default Constructor
-----
Employee ID    : 0
Employee Name  : Not Assigned
Department     : Not Assigned
Salary        : 0.0

Parameterized Constructor
-----
Employee ID    : 102
Employee Name  : Subhasri S
Department     : Development
Salary        : 30000.0
```

Constructor Explanation:

A constructor is used to initialize an object when it is created. A default constructor does not require parameters and can initialize an object with default values. A parameterized constructor accepts parameters and uses them to initialize the instance variables of an object.

TASK 3: this & static – BANK ACCOUNT

Objective

To demonstrate the use of the this keyword and a static variable while creating and managing multiple bank account objects.

Brief Explanation

This program creates a BankAccount class containing account holder name, account number, and balance. The this keyword is used to refer to the current object's instance variables and to differentiate them from constructor parameters. A static variable is used to keep track of the total number of bank accounts created. Since the static variable belongs to the class, its value is shared among all the objects.

Source Code

File Name: BankAccountDemo.java

```
class BankAccount {  
    String accountHolderName;  
    long accountNumber;  
    double balance;  
    static int totalAccounts = 0;
```

```

    BankAccount(String accountHolderName, long
accountNumber, double balance) {
        this.accountHolderName = accountHolderName;
        this.accountNumber = accountNumber;
        this.balance = balance;
        totalAccounts++;
    }

    void displayAccount() {
        System.out.println("Account Holder : " +
accountHolderName);

        System.out.println("Account Number : " +
accountNumber);

        System.out.println("Balance      : " + balance);
        System.out.println();
    }
}

public class BankAccountDemo {
    public static void main(String[] args) {
        BankAccount account1 = new
BankAccount("Subhasri S", 1001, 25000.0);
    }
}

```

```

        BankAccount account2 =new
BankAccount("Sathish", 1002, 30000.0);
BankAccount account3 =new BankAccount("Vasundra",
1003, 20000.0);

        System.out.println("Bank Account");
        System.out.println("-----");
        System.out.println();
        System.out.println("Account 1");
        System.out.println("-----");
        account1.displayAccount();
        System.out.println("Account 2");
        System.out.println("-----");
        account2.displayAccount();
        System.out.println("Account 3");
        System.out.println("-----");
        account3.displayAccount();

        System.out.println("Total Accounts : " +
BankAccount.totalAccounts);
    }

```

Output

```
● PS D:\Internova\Week_3\Task_3> javac BankAccountDemo.java
● PS D:\Internova\Week_3\Task_3> java BankAccountDemo
Bank Account
-----

Account 1
-----
Account Holder : Subhasri S
Account Number : 1001
Balance       : 25000.0

Account 2
-----
Account Holder : Sathish
Account Number : 1002
Balance       : 30000.0

Account 3
-----
Account Holder : Vasundra
Account Number : 1003
Balance       : 20000.0

Total Accounts : 3
```

Explanation of this

The `this` keyword refers to the current object of a class. It is commonly used when instance variables and constructor parameters have the same name. It helps identify the instance variable that belongs to the current object.

Explanation of static

The `static` keyword is used to define class-level members. A static variable is shared by all objects of the class. In this program, the static variable is used to maintain the total number of bank accounts created.

TASK 4: INHERITANCE – EMPLOYEE MANAGEMENT

Objective

To demonstrate inheritance in Java by creating an Employee parent class and Developer and Manager child classes.

Brief Explanation

Inheritance is an important Object-Oriented Programming concept that allows a child class to acquire the properties and methods of a parent class. In this program, the Employee class contains common properties such as name, employee ID, and salary. The Developer and Manager classes extend the Employee class and contain additional properties specific to their roles. The extends keyword is used to implement inheritance and demonstrate

Source Code

File Name: EmployeeManagement.java

```
class Employee {  
  
    String name;  
    int employeeId;  
    double salary;  
  
    Employee(String name, int employeeId, double salary)  
    {  
        this.name = name;  
        this.employeeId = employeeId;  
        this.salary = salary;  
    }  
  
    void displayEmployee() {  
        System.out.println("Name      : " + name);  
        System.out.println("Employee ID: " + employeeId);  
        System.out.println("Salary   : " + salary);  
    }  
}  
  
class Developer extends Employee {
```

```

String programmingLanguage;

Developer(String name, int employeeId, double salary,
          String programmingLanguage) {

    super(name, employeeId, salary);
    this.programmingLanguage =
programmingLanguage;
}

void displayDeveloper() {
    displayEmployee();
    System.out.println("Programming Language : " +
programmingLanguage);
}
}

class Manager extends Employee {

    int teamSize;

```

```
    Manager(String name, int employeeId, double salary,  
int teamSize) {
```

```
        super(name, employeeId, salary);  
        this.teamSize = teamSize;  
    }
```

```
void displayManager() {  
    displayEmployee();  
    System.out.println("Team Size : " + teamSize);  
}  
}
```

```
public class EmployeeManagement {  
    public static void main(String[] args) {
```

```
        Developer developer =  
            new Developer("Subhasri", 101, 45000.0,  
"Java");
```

```
        Manager manager =  
            new Manager("Sathish", 102, 55000.0, 8);
```

```
System.out.println("Employee Management");  
System.out.println("-----");  
System.out.println();
```

```
System.out.println("Developer Details");  
System.out.println("-----");  
developer.displayDeveloper();
```

```
System.out.println();
```

```
System.out.println("Manager Details");  
System.out.println("-----");  
manager.displayManager();
```

```
}
```

```
}
```

Output

```
● PS D:\Internova\Week_3\Task_4> javac EmployeeManagement.java
● PS D:\Internova\Week_3\Task_4> java EmployeeManagement
Employee Management
-----

Developer Details
-----
Name       : Subhasri
Employee ID: 101
Salary     : 45000.0
Programming Language : Java

Manager Details
-----
Name       : Sathish
Employee ID: 102
Salary     : 55000.0
Team Size  : 8
```

TASK 5: POLYMORPHISM – METHOD OVERLOADING & OVERRIDING

Objective

To demonstrate both method overloading and method overriding in Java as different forms of polymorphism.

Brief Explanation

Polymorphism is an important OOP concept that allows the same method name or interface to perform different behaviours. This program demonstrates method overloading by creating multiple calculate() methods with different parameter lists. It also demonstrates method overriding by creating Car and Bike classes that provide their own implementations of the start() method inherited from the Vehicle class.

Source Code

File Name: PolymorphismDemo.java

```
class Calculator {
```

```
    int calculate(int a, int b) {  
        return a + b;  
    }
```

```
    double calculate(double a, double b) {  
        return a + b;  
    }
```

```
    int calculate(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

```
class Vehicle {
```

```
    void start() {  
        System.out.println("Vehicle starts.");  
    }
```

```
}  
}
```

```
class Car extends Vehicle {
```

```
    @Override  
    void start() {  
        System.out.println("Car starts with a key.");  
    }  
}
```

```
class Bike extends Vehicle {
```

```
    @Override  
    void start() {  
        System.out.println("Bike starts with a self-start.");  
    }  
}
```

```
public class PolymorphismDemo {  
    public static void main(String[] args) {  
  
        Calculator calculator = new Calculator();  
  
        System.out.println("Polymorphism");  
        System.out.println("-----");  
        System.out.println();  
  
        System.out.println("Method Overloading");  
        System.out.println("-----");  
  
        System.out.println("calculate(int, int) : "  
            + calculator.calculate(10, 20));  
  
        System.out.println("calculate(double, double) : "  
            + calculator.calculate(15.5, 20.0));  
  
        System.out.println("calculate(int, int, int) : "
```

```
+ calculator.calculate(10, 20, 30));
```

```
System.out.println();
```

```
System.out.println("Method Overriding");
```

```
System.out.println("-----");
```

```
Vehicle car = new Car();
```

```
Vehicle bike = new Bike();
```

```
System.out.println("Car:");
```

```
car.start();
```

```
System.out.println("Bike:");
```

```
bike.start();
```

```
}
```

```
}
```

Output

```
● PS D:\Internova\Week_3\Task_5> javac PolymorphismDemo.java
● PS D:\Internova\Week_3\Task_5> java PolymorphismDemo
Polymorphism
-----

Method Overloading
-----
calculate(int, int) : 30
calculate(double, double) : 35.5
calculate(int, int, int) : 60

Method Overriding
-----
Car:
Car starts with a key.
Bike:
Bike starts with a self-start.
```

TASK 6: ENCAPSULATION & ABSTRACTION – BANKING SYSTEM

Objective

To implement encapsulation and abstraction in Java by creating a simple banking system.

Brief Explanation

This program demonstrates two important OOP concepts: encapsulation and abstraction. Encapsulation is implemented by keeping the account information private and accessing it through getter and setter

methods. Abstraction is demonstrated using an abstract Account class containing an abstract method and a normal method. A child class extends the abstract class and provides an implementation for the abstract method.

Source Code

File Name: BankingSystem.java

```
class SecureBankAccount {  
  
    private long accountNumber;  
    private String accountHolderName;  
    private double balance;  
  
    public long getAccountNumber() {  
        return accountNumber;  
    }  
  
    public void setAccountNumber(long accountNumber)  
    {  
        this.accountNumber = accountNumber;  
    }  
  
    public String getAccountHolderName() {  
        return accountHolderName;  
    }  
  
    public void setAccountHolderName(String  
accountHolderName) {  
        this.accountHolderName = accountHolderName;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

```

    }

    public void setBalance(double balance) {
        this.balance = balance;
    }
}

abstract class Account {

    abstract void calculateInterest();

    void displayAccountType() {
        System.out.println("Account Type : Savings
Account");
    }
}

class SavingsAccount extends Account {

    @Override
    void calculateInterest() {
        System.out.println("Interest calculation
completed.");
    }
}

```

```
public class BankingSystem {  
  
    public static void main(String[] args) {  
  
        SecureBankAccount account = new  
SecureBankAccount();  
  
        account.setAccountNumber(1001);  
account.setAccountHolderName("Subhasri");  
account.setBalance(25000.0);  
  
        System.out.println("Banking System");  
System.out.println("-----");  
System.out.println();  
  
        System.out.println("Account Details");  
System.out.println("-----");  
  
        System.out.println("Account Number : "  
        + account.getAccountNumber());  
  
        System.out.println("Account Holder : "  
        + account.getAccountHolderName());  
  
        System.out.println("Balance      : "  
        + account.getBalance());  
    }  
}
```

```
account.setBalance(30000.0);
```

```
System.out.println();
```

```
System.out.println("Updated Balance");
```

```
System.out.println("-----");
```

```
System.out.println("Balance      : "  
    + account.getBalance());
```

```
System.out.println();
```

```
System.out.println("Account Functionality");
```

```
System.out.println("-----");
```

```
SavingsAccount savingsAccount = new  
SavingsAccount();
```

```
savingsAccount.displayAccountType();
```

```
savingsAccount.calculateInterest();
```

```
}
```

```
}
```

Output

```
● PS D:\Internova\Week_3\task_6> javac BankingSystem.java
● PS D:\Internova\Week_3\task_6> java BankingSystem
Banking System
-----

Account Details
-----
Account Number : 1001
Account Holder : Subhasri
Balance        : 25000.0

Updated Balance
-----
Balance        : 30000.0

Account Functionality
-----
Account Type : Savings Account
Interest calculation completed.
```